

1. Introducción. Motivo por el que se ha optado por hacer esta aplicación

El proyecto descrito consiste en una **aplicación de gestión basada en una Pokédex** cuyo propósito es **almacenar y registrar nuevos Pokémon** y **permitir la realización de combates entre ellos**.

Desde un punto de vista de ingeniería de software, este tipo de aplicación es representativo porque combina dos necesidades técnicas frecuentes:

- **Gestión de información persistente:** alta (registro) y almacenamiento de entidades (Pokémon) que deben permanecer disponibles entre ejecuciones.
- **Lógica de negocio no trivial:** implementación de un **motor de combate** que, a partir de los datos de dos Pokémon, ejecute reglas de enfrentamiento y determine un resultado.

1.1. Alcance real disponible y limitaciones del análisis (importante)

Para elaborar una memoria “de proyecto” con trazabilidad completa (tecnologías usadas, clases reales, esquema real de base de datos, pantallas exactas, evidencias de pruebas), es necesario disponer del **código fuente**, configuración y/o documentación técnica.

En el material entregado como proyecto (archivo ZIP **Proyecto.zip**) únicamente se observa una **estructura de carpetas**(p. ej. `PokeDexManager`, `.vs`, `bin/Debug`, `obj`, `Properties`, `Resources`, `Help`, `es-ES`) **sin ficheros de código ni recursos**. Esto impide:

- Verificar el **lenguaje**, framework o versión exacta (aunque la estructura es *compatible* con un proyecto de Visual Studio/.NET, no es demostrable al 100% sin ficheros).
- Determinar el **modelo de datos real** (si existe base de datos, si es relacional, si es fichero local, etc.).
- Confirmar el **diseño de clases** implementado.
- Identificar **interfaces/pantallas** reales.
- Confirmar el **algoritmo de combate** (reglas, estadísticas, aleatoriedad, turnos, tipos, etc.).
- Aportar **evidencias de pruebas ejecutadas** (informes, logs, capturas, resultados automáticos).

Por ello, esta memoria:

- **Documenta lo explícitamente indicado** (registro/almacenamiento de Pokémon y combates).
- **Señala vacíos técnicos** que deben concretarse.
- Incluye un **diseño técnico mínimo coherente a nivel de propuesta/derivación de requisitos** cuando es imprescindible para explicar el funcionamiento (por ejemplo,

qué datos mínimos necesita un combate). Estas partes se redactan como *recomendación o diseño mínimo deducido*, no como “implementación confirmada”.

2. Objetivos de la aplicación

2.1. Objetivo general

Desarrollar una aplicación capaz de **gestionar un repositorio persistente de Pokémon definidos/registrados por el usuario** y ejecutar **combates entre Pokémon registrados**, mostrando el resultado del enfrentamiento.

2.2. Objetivos específicos (derivados del enunciado)

- 1 **Registro de Pokémon:** permitir la creación de nuevos Pokémon dentro de la Pokédex de la aplicación.
- 2 **Almacenamiento persistente:** asegurar que los Pokémon registrados queden almacenados de forma que puedan recuperarse posteriormente.
- 3 **Selección de combatientes:** permitir seleccionar Pokémon registrados para enfrentarlos.
- 4 **Ejecución de combates:** implementar una lógica que procese un combate entre dos Pokémon y determine un resultado (ganador/derrota/empate, según reglas definidas).
- 5 **Presentación del resultado:** mostrar al usuario el resultado del combate (y, si aplica, el detalle del desarrollo del combate).

2.3. Objetivos técnicos recomendados (no confirmados, pero necesarios para robustez)

Aunque no se indican explícitamente, para que el sistema sea mantenible y consistente, suelen ser necesarios:

- **Validación de datos** (evitar registros incompletos o inconsistentes).
- **Integridad de persistencia** (identificadores únicos, control de duplicados, reglas de borrado si existiera).
- **Separación de responsabilidades** (UI vs lógica de combate vs persistencia).
- **Trazabilidad de combates** (si se requiere auditoría/histórico).

3. Requisitos de funcionamiento de la aplicación

3.1. Requisitos funcionales (confirmados por el enunciado)

RF-01. **Registrar, editar y eliminar Pokémon** en la Pokédex de la aplicación.

RF-02. **Almacenar** los Pokémon registrados (persistencia).

RF-03. **Realizar combates** entre Pokémon registrados.

3.2. Requisitos funcionales implícitos (necesarios para RF-03, pero no definidos)

Para ejecutar un combate, la aplicación debe definir al menos:

- **Datos mínimos por Pokémon** para poder calcular daño/ganador:
 - **Puntos de vida.**
 - **Puntos de ataque.**
 - **Puntos de defensa.**
- **Reglas de combate:**
 - Cálculo de daño((Ataque – Defensa) – Puntos de vida).
 - Condición de victoria (El pokemon con menos vida después del cálculo de daño pierde).
 - Empates (No se guarda en el registro de combates).

3.3. Requisitos no funcionales (recomendados; no verificables)

RNF-01. **Persistencia fiable:** los datos no deben corromperse ante cierres inesperados.

RNF-02. **Consistencia:** evitar Pokémon con valores fuera de rango (Daño negativo, etc.).

RNF-03. **Usabilidad:** flujo claro para registrar Pokémon y lanzar combates, con mensajes de validación.

RNF-04. **Mantenibilidad:** estructura modular con separación de capas.

4. Diagrama de casos de uso

4.1. Actores

- **Usuario:** persona que interactúa con la aplicación para registrar Pokémon y ejecutar combates.

4.2. Casos de uso mínimos (derivados del enunciado)

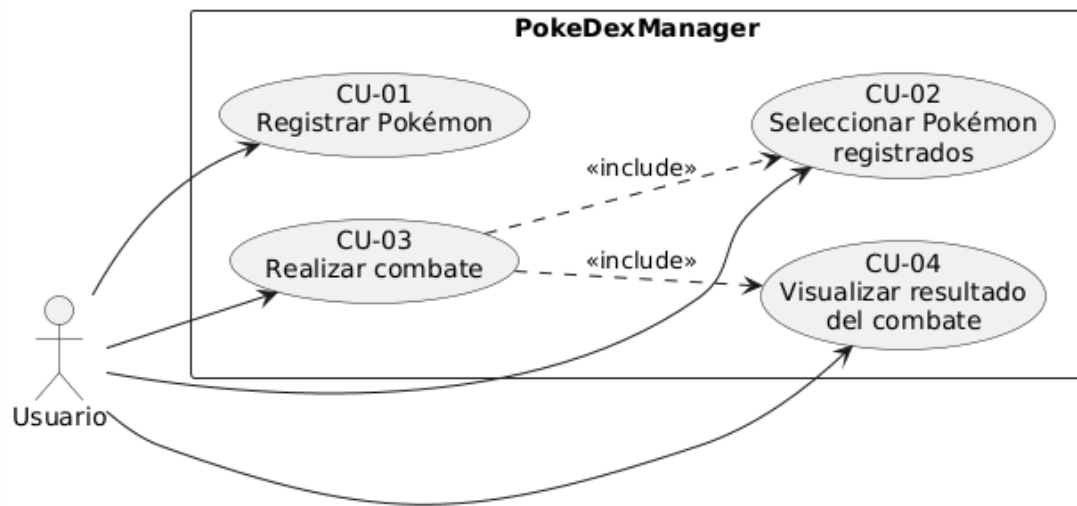
- **CU-01 Registrar Pokémon:** alta de un nuevo Pokémon en la Pokédex.
- **CU-02 Consultar/Seleccionar Pokémon registrados:** selección de Pokémon existentes para operaciones (al menos para combatir).

- **CU-03 Realizar combate:** elegir dos Pokémon y ejecutar la simulación del combate.
- **CU-04 Visualizar resultado del combate:** mostrar ganador y/o detalles.

CU-02 se considera necesario para CU-03; si no existe “consulta”, debe existir un mecanismo alternativo de selección (introducir ID/nombre), que también forma parte del caso de uso.

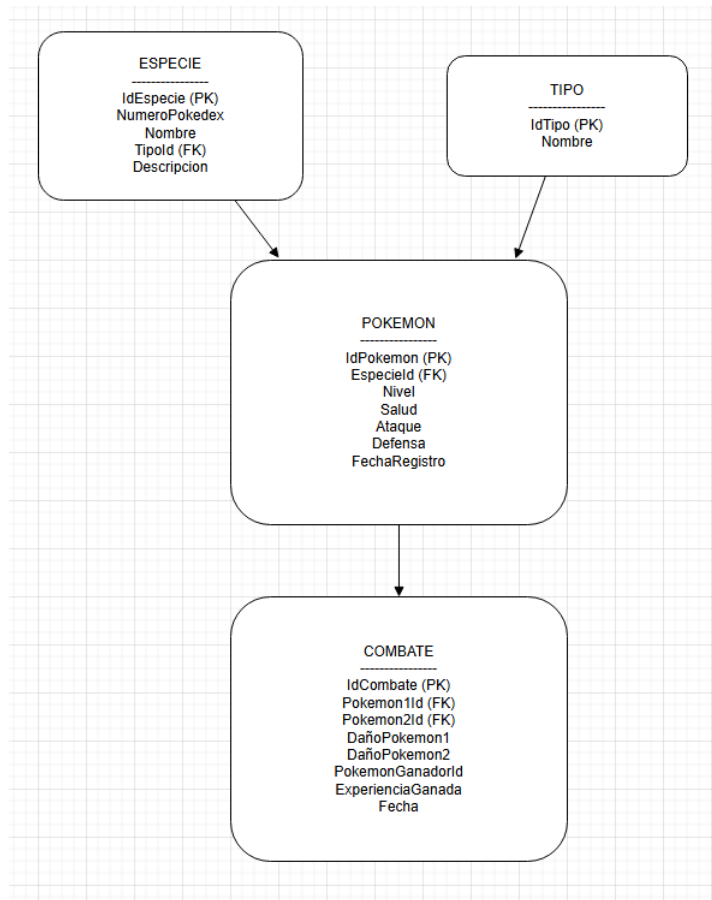
4.3. Representación del diagrama

El siguiente diagrama es una representación textual estándar (no implica que el proyecto lo haya implementado así; describe los casos de uso mínimos):



5. Diseño de la aplicación

5.2. Diseño de base de datos

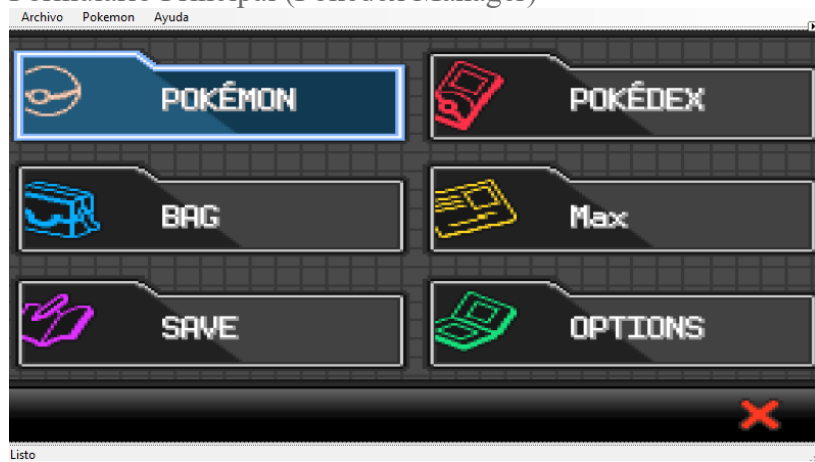


5.2. Diseño de las clases e interfaces utilizadas

SplashScreen



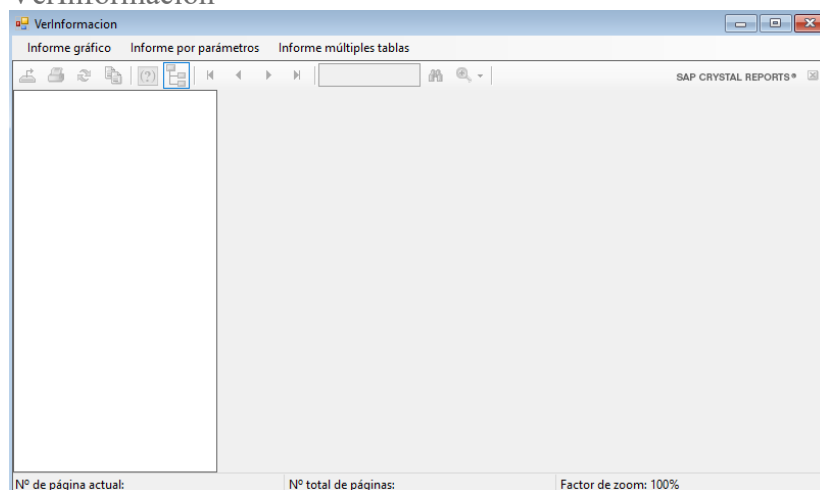
Formulario Principal (Pokedex Manager)



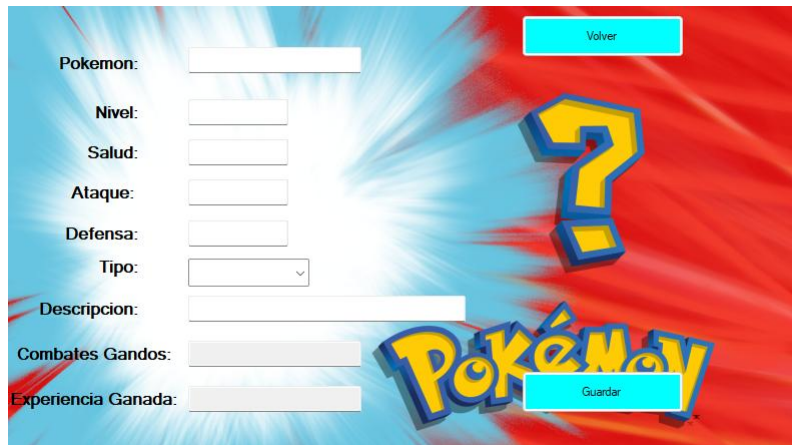
NuevoPokemon



VerInformacion



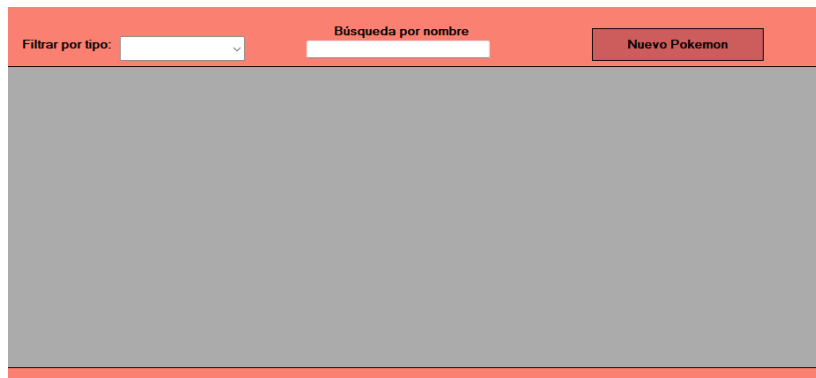
VerPokemon



UcCombates



UcPokedex



6. Funcionamiento de la aplicación

Al cargar la SplashScreen se inicia el formulario principal en el que hay varias opciones:

- Archivo: Permite salir de la aplicación.
- Ver Pokedex: Abre el User Control UcPokedex, el cual contiene la lista de pokemons permitiendo crear, editar y eliminar pokemons a la vez que filtrar por tipos y buscar por nombre. Al hacer click en un pokemon te lleva a la vista VerPokemon en la que aparece toda la información del pokemon. Al hacer click en Crear Pokemon se abre la vista NuevoPokemon en la que introduces los datos del nuevo pokemon.

- Registrar Combate: Abre el User Control UcCombates, el cual contiene la lista de combates realizados entre dos pokemons existentes en la BBDD, permite realizar nuevos combates seleccionando dos pokemons diferentes, el ganador se decide con la siguiente fórmula ((Ataque – Defensa) – Puntos de vida) siendo el que tenga más vida el ganador. El ganador consigue un numero aleatorio de experiencia y suma un combate ganado.
- Acerca de: Abre la documentación de la aplicación.
- Informes: Abre la vista VerInformación que permite ver diferentes informes sobre los datos de la aplicación.
- Guía de usuario: Abre un manual de ayuda para que el usuario conozca todas las posibilidades de la aplicación.

7. Pruebas

NuevoPokemon

Especie Pokemon: prueba

Nivel: 50

Salud: 6

Ataque: 4

Defensa: 5

Tipo: Acero

Descripcion: fgafdafd

Crear

Archivo Pokemon Ayuda

Filtrar por tipo: Roca Búsqueda por nombre pr Nuevo Pokemon

	NumeroPokedex	Nombre	Tipo	FechaRegistro
▶	3	prueba	Roca	02/03/2026
*				

Lista Pokémon cargados: 2

Archivo Pokemon Ayuda

Charmander Pelea pruna

	IdCombate	Pokemon1	Pokemon2	Ganador	DañoPokemon1	DañoPokemon2	ExperienciaGanac	Fecha
▶	1	Charmander	pruna	Charmander	9	0	11	02/03/2026 22:...
*								

Listo Pokémon cargados: 2

Pokemon: Charmander

Nivel: 12

Salud: 10

Ataque: 10

Defensa: 10

Tipo: Fuego

Descripcion: si

Combates Ganados: 1

Experiencia Ganada: 11

Volver

Verinformación

Informe gráfico Informe por parámetros Informe múltiples tablas

SAP CRYSTAL REPORTS®

Informe principal

Informe Gráfico

Pokemons por Tipos

02/03/2026	Fuego	Numero Pokedex	Nombre	FechaRegistro
		1	Charmander	10/02/2026
		Total de pokemons: 1		
Roca	Numero Pokedex	Nombre	FechaRegistro	
	3	pruna	02/03/2026	
		Total de pokemons: 1		
Pokemons totales:		2		

Nº de página actual: 1 Nº total de páginas: 1 Factor de zoom: 100%

8. Dificultades que han surgido en el desarrollo de la aplicación y su solución

Problema: Incoherencia de datos entre pokemon y combates.
Solución: Editar varias veces la BBDD.

También algún problema menor con solución fácil.

9. Posibles ampliaciones de la aplicación

- Modelo de tipos y efectividades

- Doble tipo y multiplicadores de daño por efectividad.

2 Movimientos/habilidades

- Conjunto de ataques por Pokémon con PP, potencia, precisión.

3 Combate por turnos

- Los combates continúan hasta que uno de los dos pokemons se queden sin vida.

4 Perfiles de usuario

- Separar colecciones por usuario.

10. Conclusión

La aplicación propuesta se centra en dos capacidades: **gestión persistente de Pokémon (registro/almacenamiento)** y **ejecución de combates** entre Pokémon registrados.

Desde el punto de vista técnico, el éxito del proyecto depende especialmente de:

- Definir un **modelo de datos** mínimo y consistente para Pokémon.
- Especificar de forma inequívoca el **algoritmo de combate**.
- Implementar una persistencia robusta y un diseño modular que desacople UI, dominio y datos.

11. Referencias consultadas

[PokeApi](#)

[ChatGPT](#)